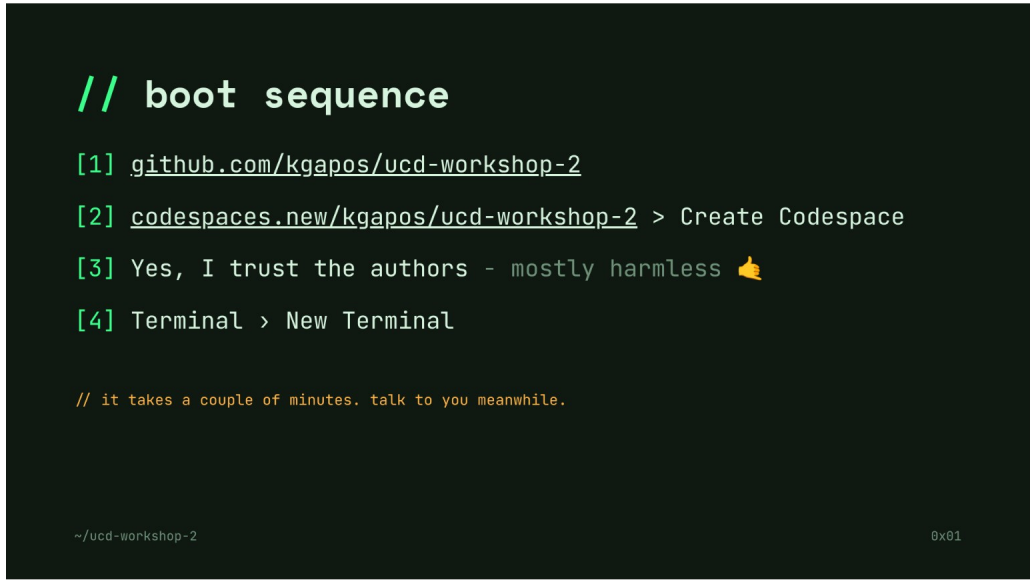
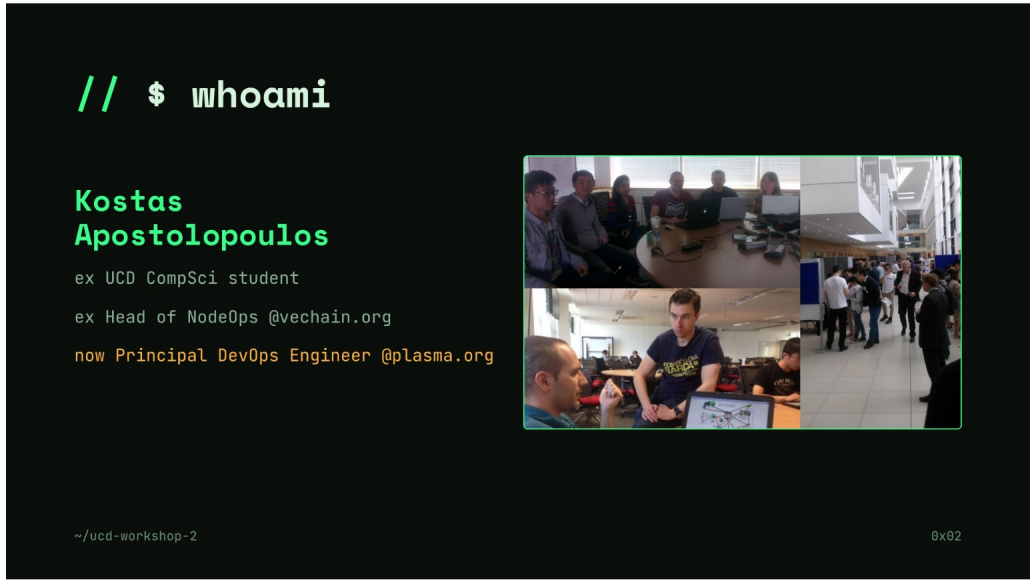


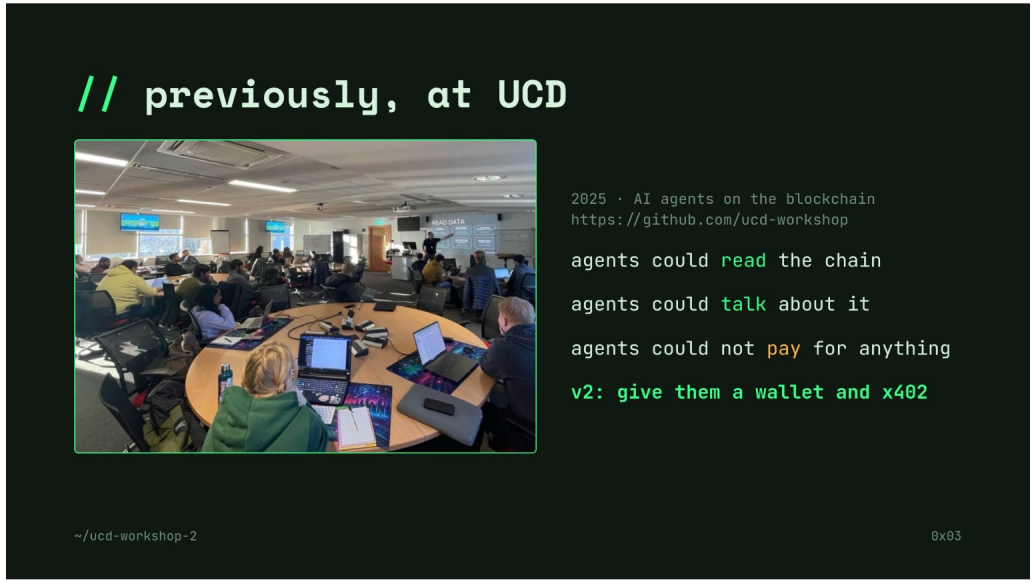
[~30s] Welcome, quick intro of yourself. Say: today you build an API that charges per request, and a client that pays it automatically. No sign-ups, no API keys, no credit cards. Point at the curl line: by the end, getting a 402 back will be the normal, happy path. Reassure: everything runs on a testnet. The money is fake, so nobody can lose anything. Transition: before theory, get your Codespace booting, because it takes a couple of minutes.



[~1 min] Part 0. Keep this slide up until most hands are done. Do: everyone opens github.com/kgapos/ucd-workshop-2, clicks the Open in GitHub Codespaces button, then Create codespace. Wait for the files to appear, then approve the trust prompt. That is when dependencies install. Open a terminal: Terminal, New Terminal. Cost question will come up: a personal account gets about 60 hours a month free on this machine size. Stuck? No GitHub account or Codespaces greyed out: they can pair with a neighbour. Transition: while it builds, a bit of history.



[~45s, while Codespaces build] Who am I. Point at the photos: these are from my own time at UCD as a CompSci student, same building, same late nights, and a poster session in the atrium. Career in one breath: Head of NodeOps at VeChain, now Principal DevOps Engineer at Plasma. Transition: and this is not my first workshop here.



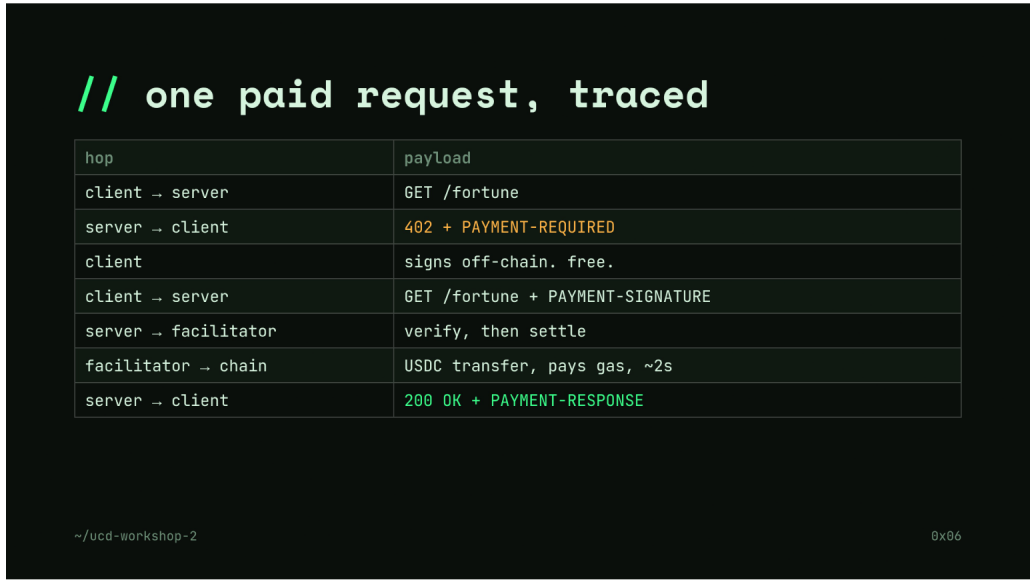
[~45s] Last year, same rooms: we built AI agents on top of the VeChain SDK and ElizaOS. They could read blocks, balances and transfers, and chat about what they found. Code is still at github.com/kgapos/ucd-workshop. The catch: the moment an agent wants to do something useful in the world, it hits a paywall. Data feeds, APIs, compute, other agents. It cannot sign up for an account, cannot pass a captcha, and nobody gives it a credit card. So agents were smart but broke. This year is the sequel: we give programs and agents a wallet, and teach HTTP how to ask for money. Transition: which starts with a status code nobody has used for 29 years.



[~45s] Ask the room: who knows 404? 500? 418, I am a teapot? Now, who has ever seen a 402? Say: HTTP has had a status code for payments since 1997. The spec reserved it for future use. For almost thirty years nobody agreed what a client should actually do when it got one. So payments ended up happening around HTTP, not inside it: accounts, keys, cards, subscriptions. Transition: here is what buying one API call looks like today.



[~45s] Left: the usual dance to use a paid API. Five steps before your first request, and most of them need a human. Right: x402. Ask, get told the price, sign, retry, done. One round trip extra. Key point: the only credential is a key pair. A program can hold one, so programs and AI agents can pay without a human filling in forms. x402 is an open protocol, started by Coinbase, now an open foundation. Not tied to one company or one chain. Transition: let us trace one request in detail.



[~1.5 min] The whole protocol on one slide. Walk it row by row. 1. Client asks for /fortune like any normal request. 2. Server answers 402 with a PAYMENT-REQUIRED header: price, token, network, and who to pay. Base64 JSON; they will decode it themselves in Part 2. 3. Client signs a payment authorisation with its private key. Local, off-chain, free. 4. Client retries the same request with a PAYMENT-SIGNATURE header. 5. Server hands it to the facilitator: verify it, then settle it. 6. Facilitator submits the USDC transfer on-chain and pays the gas. About 2 seconds on Base. 7. Server returns 200, the content, and a PAYMENT-RESPONSE receipt with the transaction hash. Point out: the client never sends a blockchain transaction and never needs ETH for gas. Transition: so who is this facilitator, and should you trust it?



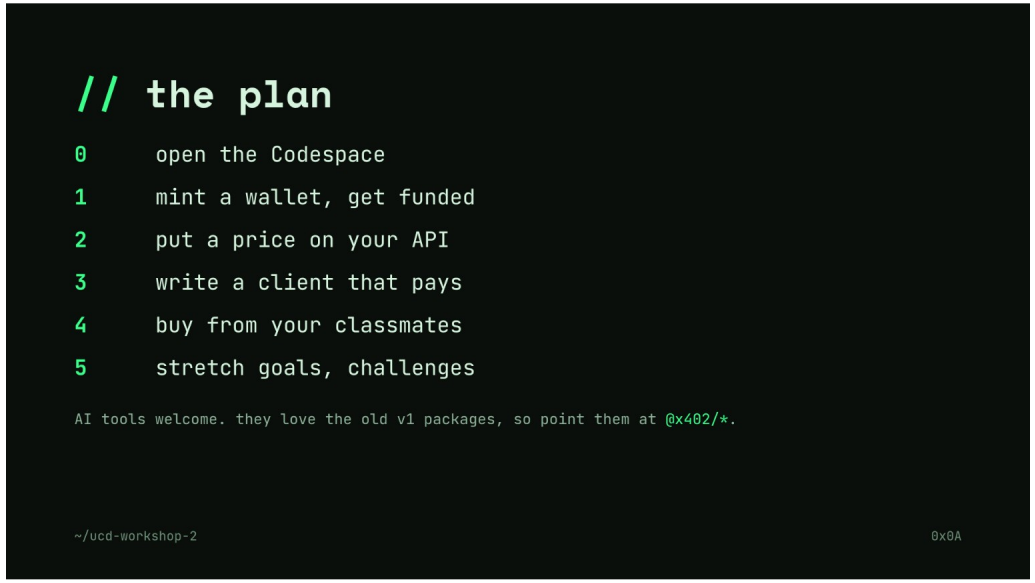
[~1 min] The facilitator is a helper service the server talks to. Two jobs: verify and settle. What the client signs is an EIP-3009 transferWithAuthorization: exactly this amount, to this address, valid for a few minutes, with a one-time nonce. So the facilitator can submit it and pays the gas, but it cannot change the amount, redirect it, or replay it. Worst case it refuses or goes down. Trusted for liveness, not custody. Today we use a class facilitator on my homelab, facilitator.kgapos.com, because the public one struggles with 40 people at once. Anyone can run one. Challenge 12 asks them to, and to think about what a dishonest one could do. Transition: why stablecoins and not, say, a credit card?



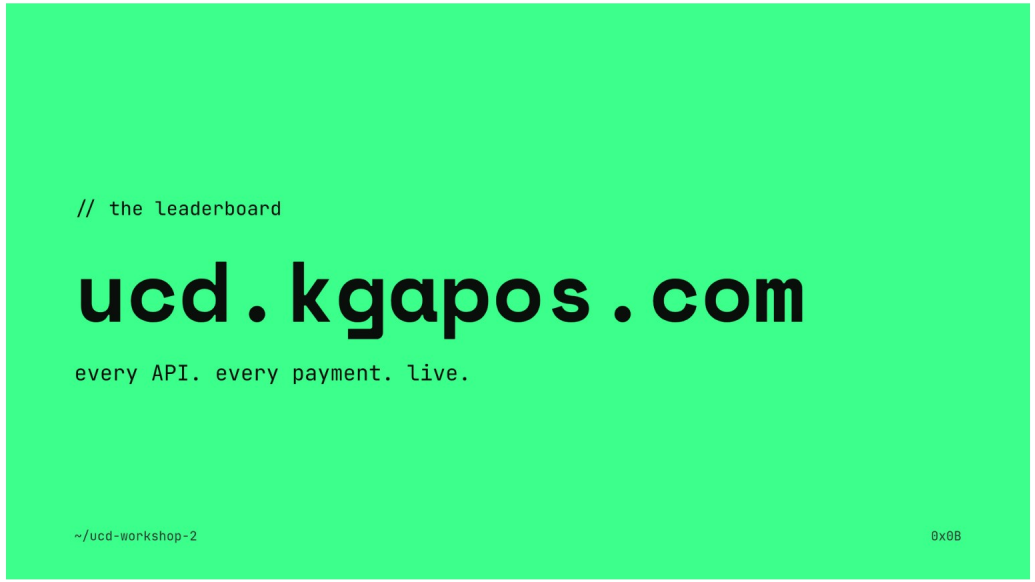
[~1 min] Five reasons, one line each. Stable: a price of a tenth of a cent stays a tenth of a cent. A volatile token makes pricing an API a guessing game. Tiny: USDC has 6 decimals, so $0.001 is 1000 base units. Card fees, around 30 cents plus a percentage, kill anything under a few cents. Fast and final: about 2 seconds, no chargebacks. No accounts: a program needs a key pair, not a bank account or a contract with every provider. Programmable: a payment is just a signed message, so code decides when and how much to pay. Be honest about the trade-offs (amber line): every payment is public, lose the key and you lose the funds, and the issuer can freeze addresses. Good discussion material. Transition: where all this runs today.



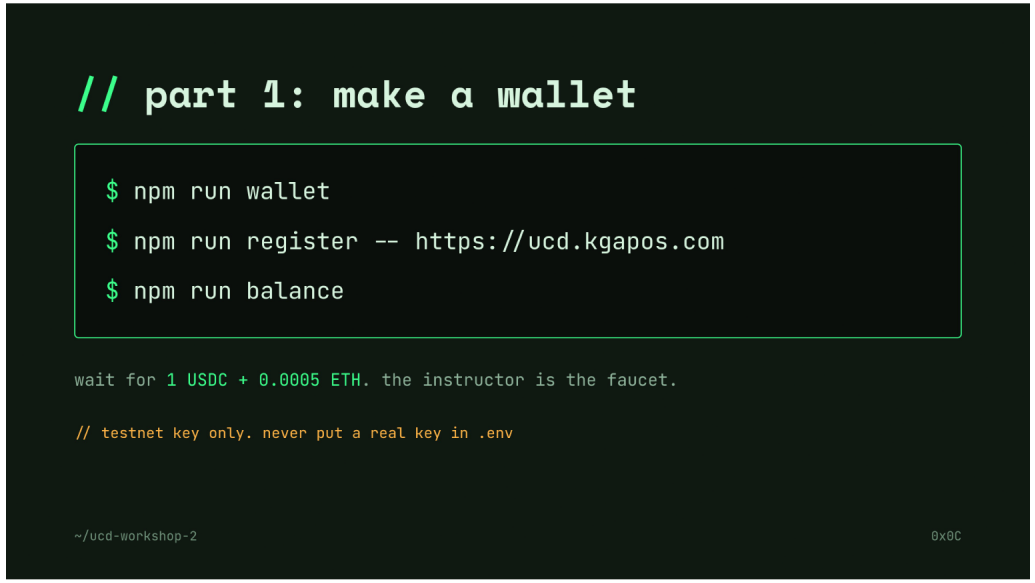
[~30s] Base Sepolia: a public Ethereum layer-2 testnet, network id eip155:84532. Why this one: x402 supports it, Circle's testnet USDC on it supports EIP-3009, blocks every 2 seconds, and a public explorer. The tokens are worth nothing, so break things freely. Everything is public: you will open your own payments on sepolia.basescan.org and see who paid the gas. Transition: here is the plan for the next two hours.



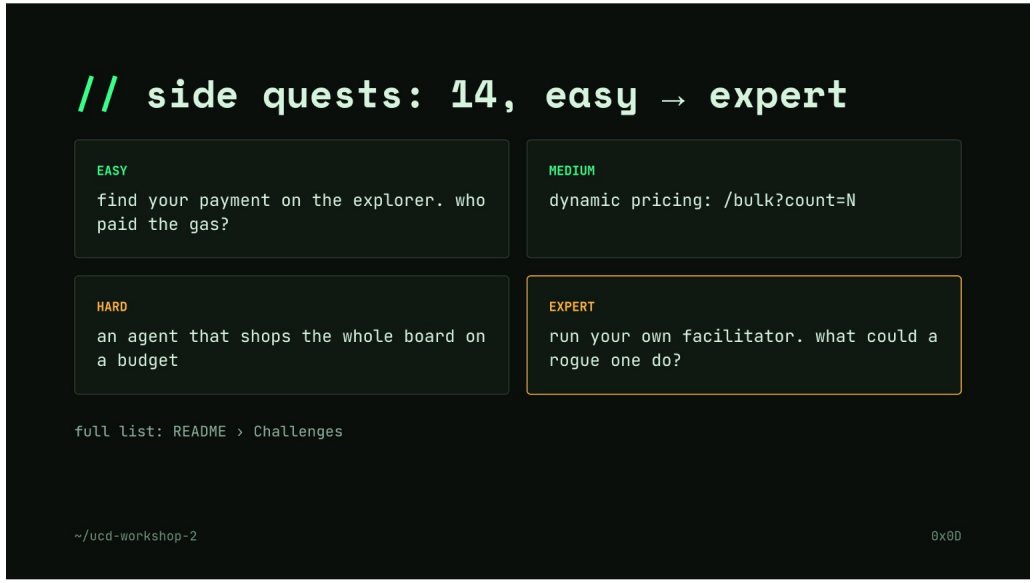
[~45s] Six parts, all in the README. Work at your own pace. Parts 2 and 3 have TODOs in starter/server.js and starter/client.js, with hints under each part. Part 4 is the fun one: you buy from each other, and the board shows who earns the most. Safety net: about an hour in, I push the reference solution. git pull and you are unblocked. AI tools are welcome, but they tend to suggest the old v1 packages like x402-express. This repo uses the v2 packages under @x402. Point them at docs.x402.org. Raise a hand any time. Transition: first, the board.



[~20s] Do: open ucd.kgapos.com on the projector and leave it there for the rest of the session. Say: keep it open in a tab. It lists every registered API, whether it is up, paid requests, USDC earned and your balance. It is also how I find your address to send you testnet USDC. Labels to know: not funded means wait for me; port is private or port not forwarded means check the Troubleshooting table. Transition: let us get you on it.



[leave up ~5 min] Part 1. Three commands. npm run wallet: creates a private key, saves it to .env, prints your address. Testnet only, and .env is git-ignored. npm run register: sends your name, address and API URL to the board, and saves the class facilitator to .env. Instructor, once most names are on the board: npm run fund -- https://ucd.kgapos.com/apis.json --dry-run, then without --dry-run. Re-run it for late arrivals; nobody is paid twice. npm run balance: wait until it shows 1 USDC and 0.0005 ETH, then open the explorer link it prints. Then: everyone continues with Part 2 in the README at their own pace.



[show when the first people finish Part 4, ~30s] Finished early? Fourteen challenges in the README, ordered easy to expert. Easy: find your own payment on the explorer and spot that the facilitator, not you, paid the gas. Medium: dynamic pricing, /bulk?count=N. Hard: an agent that shops the whole board without paying anyone twice. Expert: run your own facilitator and argue what a rogue one could and could not do. Also fun: challenge 13, a reseller that buys from a classmate and resells at a markup.



[~1 min, last 5 minutes of the session] Wrap up. Do: show the board one last time and congratulate the top earners. The quote: from 1993, the idea that privacy and freedom online come from people building tools, not from asking permission. Today you wrote some code that moves money with no bank and no account. That is the spirit. Everything stays in the repo and the solution is on main. Codespaces stop when idle and they can delete them from github.com/codespaces. Find me at kgapos.com, or as kgapos on GitHub, X and LinkedIn. Plug: Plasma is hiring, plasma.org/careers, come talk to me after if you enjoyed today. Thank the hosts, UCD and Plasma. $ exit 0.